

Midterm Project: Production-Ready Observability for Your Custom Web App

Form of Submission: Individual project report (PDF), Presentation slides, and Oral Defense

Objective

To bridge the gap between Software Engineering and Site Reliability Engineering. You must take the existing web application you developed in your Web Frontend and Backend courses, containerize it, establish reliability targets (SLOs), and build a complete SRE observability stack to monitor it in real-time.

Scenario: "Bring Your Own App"

You are no longer just the developer of your application; you are now its Lead SRE. You are not allowed to use a simple "Hello World" template for this midterm.

Your mission is to take the actual source code of the website you built for your other university courses, package it using Docker, mathematically define what a "healthy" state means for your specific business logic, and build the dashboards to prove your system is reliable.

Requirements

Step 1: Containerizing Your Custom Architecture (Docker Compose)

- Take the source code of your existing custom web application.
- Create a Dockerfile for your frontend (e.g., React, Vue, or Vanilla JS).
- Create a Dockerfile for your backend (e.g., Node.js, Python, Java).
- Create a docker-compose.yml that deploys:
 - Frontend service
 - Backend service
 - Database (PostgreSQL, MongoDB, etc.)
 - Observability Stack (Prometheus, Grafana, Node Exporter)

BONUS (+10 Points): Instead of using standard docker-compose up, deploy your entire stack using **Docker Swarm** (docker stack deploy). You must demonstrate the swarm cluster and service replicas.

Deliverable: Dockerfiles, docker-compose.yml, and screenshot showing your website running.

Step 2: Custom Reliability Targets (SLI, SLO, Error Budget)

- Define 2 SLIs specific to your app
- Set realistic SLOs
- Calculate monthly Error Budget

Deliverable: Explanation + calculations in report

Step 3: Custom Monitoring & Dashboards (Grafana)

- Connect Prometheus to Grafana
- Create dashboard with Golden Signals
- Show SLO compliance

Deliverable: Dashboard screenshots

Step 4: Alerting Validation (Prometheus)

- Create 2 alerts (Warning + Critical)
- Trigger one alert manually

Deliverable: alert_rules.yml + FIRING alert screenshot

Submission Instructions

PDF Report: Full documentation

Presentation Slides: 5–7 slides

Oral Defense: Live demo + Q&A

Grading Rubric (100 Points Total)

Criteria	Description	Points
Custom Containerization	Dockerfiles and docker-compose.yml working correctly	15
SLO & Error Budget Math	Correct SLIs, SLOs, and calculations	15
Observability (Grafana)	Dashboard with Golden Signals	20
Alerting & Validation	Working alerts and triggered incident	20
Defense & Presentation	Explanation, slides, and	30

	answers	
BONUS: Docker Swarm	Successfully orchestrating the app and monitoring stack using Docker Swarm.	+10